

FusionCore: A 23-State Unscented Kalman Filter for IMU, Wheel Encoder, GPS, and Visual SLAM Fusion in ROS 2

Manan Kharwar

Independent Researcher, Hamilton, ON, Canada

manan.kharwar@outlook.com

Abstract—We present FusionCore, an open-source ROS 2 sensor fusion package that fuses IMU, wheel encoder odometry, GPS, and Visual SLAM (VSLAM) pose into a single 100 Hz odometry stream using a 23-state Unscented Kalman Filter (UKF). The 23rd state is an online estimate of the wheel encoder’s systematic yaw rate bias, which is identified through GPS heading cross-covariance and subtracted during GPS blackouts to reduce heading drift in coast mode. FusionCore also estimates gyroscope and accelerometer biases as explicit filter states, handles GPS natively in ECEF without a separate coordinate projection node, applies per-sensor Mahalanobis chi-squared outlier gating calibrated to measurement degrees of freedom, and adapts sensor noise covariance automatically from the innovation sequence. VSLAM pose fusion enables GPS-denied operation with any visual odometry or SLAM system, including automatic recovery from map reinitialization. We evaluate against `robot_localization` on twelve full-length sequences (55–92 min each) from the NCLT public dataset [1]. FusionCore achieves lower Absolute Trajectory Error (ATE) on ten of twelve sequences, with improvements ranging from $1.2\times$ to $22.2\times$ on winning sequences. The `robot_localization` UKF diverges numerically on all twelve sequences. FusionCore is available at <https://github.com/manankharwar/fusioncore> under the Apache 2.0 license.

Index Terms—sensor fusion, unscented Kalman filter, ROS 2, GPS, IMU, wheel odometry, visual SLAM, state estimation

I. INTRODUCTION

Accurate pose estimation is a prerequisite for autonomous robot navigation. Fusing IMU, wheel odometry, and GPS is a well-studied problem, yet practical deployment remains difficult. IMU gyroscope and accelerometer biases drift continuously and must be estimated online. GPS arrives at low rate with intermittent quality degradation. Wheel odometry provides high-rate relative motion but accumulates error on slippery surfaces. Critically, the wheel encoder’s own yaw rate bias (a systematic offset in differential drive angular velocity estimation) is rarely treated as a filter state, leaving it unobservable and free to accumulate heading error during GPS blackouts.

`robot_localization` [2] is the dominant ROS package for this task, with widespread adoption across academic and commercial robots. It offers an EKF and UKF, handles multiple simultaneous sensor inputs, and has extensive

documentation. However, several architectural choices limit its utility for modern deployments: gyro and accelerometer biases are not part of the filter state; GPS requires a separate `navsat_transform` node projecting coordinates through UTM, which introduces zone boundary failures; outlier rejection uses manually tuned Mahalanobis scalar thresholds that do not account for measurement dimensionality; and noise covariance is fixed at configuration time.

FusionCore addresses each of these limitations in a single ROS 2 lifecycle node with no additional coordinators. The contributions of this work are:

- 1) A 23-state UKF formulation with continuous online gyroscope bias, accelerometer bias, and wheel encoder yaw rate bias estimation.
- 2) ECEF-native GPS fusion eliminating UTM projection and its associated zone boundary failures.
- 3) Per-sensor Mahalanobis chi-squared outlier gating with thresholds calibrated per measurement path and degrees of freedom.
- 4) Automatic noise covariance adaptation from a sliding innovation window using exponential moving average.
- 5) Automatic zero-velocity updates (ZUPT) that suppress bias drift during stationary periods.
- 6) IMU ring-buffer retrodiction for GPS and VSLAM delay compensation.
- 7) Visual SLAM pose fusion for GPS-denied operation, accepting `nav_msgs/Odometry` from any visual odometry or SLAM system, with automatic recovery from map reinitialization.
- 8) GPS velocity fusion via a configurable topic accepting `nav_msgs/Odometry` for cross-sensor slip detection.
- 9) Radar Doppler ego-velocity fusion via a configurable body-frame velocity topic for GPS-aided platforms with 4D imaging radar.
- 10) A quantitative evaluation on twelve full-length sequences of the NCLT public dataset against `robot_localization` EKF and UKF.

II. RELATED WORK

Kalman filtering for inertial-GPS fusion has been studied for decades [3]. Loosely coupled integration treats GPS as position measurements, while tightly coupled integration fuses raw pseudoranges. FusionCore implements loosely coupled fusion using `sensor_msgs/NavSatFix` or, optionally, `gps_msgs/GPSFix` for RTK-capable receivers.

`robot_localization` [2] is the canonical ROS implementation. It supports EKF and UKF with arbitrary sensor combinations. Known limitations include UTM zone boundary crashes [12], numerical UKF instability under aggressive maneuvers [11], and the two-node GPS architecture requiring `navsat_transform`.

MSF [4] and ROVIO [5] target visual-inertial odometry (VIO). `imu_filter_madgwick` [6] provides complementary filter orientation estimation but not full state estimation with GPS. GTSAM [7] and Ceres [8] offer factor-graph optimization but require batch or sliding-window formulations not well-suited to online 100 Hz output. FusionCore targets the practical ROS 2 use case: a single lifecycle node that a practitioner can drop into an existing Nav2 stack with a YAML configuration file.

III. SYSTEM ARCHITECTURE

FusionCore is structured as two packages. `fusioncore_core` is a pure C++17 library with no ROS dependency implementing the UKF, measurement models, and sensor abstractions. `fusioncore_ros` wraps it as a ROS 2 lifecycle node, managing subscriptions, TF broadcasting, and lifecycle state transitions. This separation allows standalone use of the filter library in non-ROS environments.

The node subscribes to `sensor_msgs/Imu` (primary IMU), an optional second IMU via `imu2.topic`, `nav_msgs/Odometry` (wheel encoders, VSLAM pose, GPS velocity, and radar Doppler in body frame), and GPS position as either `sensor_msgs/NavSatFix` or `gps_msgs/GPSFix` (selectable at runtime). The `GPSFix` interface unlocks RTK_FLOAT fix-type reporting and receiver-native HDOP/VDOP fields unavailable in the `NavSatFix` message. All sensor inputs except the primary IMU are optional; FusionCore supports any combination. It publishes `nav_msgs/Odometry` at 100 Hz on `/fusion/odom` and broadcasts the `odom` $\rightarrow$ `base_link` transform.

IV. FILTER FORMULATION

A. State Vector

The filter maintains a 23-dimensional state vector:

$$\mathbf{x} = [\mathbf{p} \quad \mathbf{q} \quad \mathbf{v} \quad \boldsymbol{\omega} \quad \mathbf{a} \quad \mathbf{b}_g \quad \mathbf{b}_a \quad b_{ewz}]^T \quad (1)$$

where $\mathbf{p} \in \mathbb{R}^3$ is position in ENU, $\mathbf{q} = [q_w, q_x, q_y, q_z]$ is unit quaternion orientation, $\mathbf{v} \in \mathbb{R}^3$ and $\boldsymbol{\omega} \in \mathbb{R}^3$ are body-frame linear and angular velocity, $\mathbf{a} \in \mathbb{R}^3$ is body-frame linear acceleration, $\mathbf{b}_g \in \mathbb{R}^3$ is gyroscope bias, $\mathbf{b}_a \in \mathbb{R}^3$ is accelerometer bias, and $b_{ewz} \in \mathbb{R}$ is the wheel encoder angular velocity Z bias.

Gyroscope and accelerometer biases are first-class filter states with dedicated process noise parameters q_{gyro_bias} and q_{accel_bias} . This allows the filter to converge on sensor-specific bias values during operation without a separate calibration phase.

The 23rd state, b_{ewz} , is the systematic yaw rate bias of the wheel encoder. In a differential drive robot, this arises from wheel radius mismatch, mechanical asymmetry, and surface irregularities; it is a near-constant offset in the reported ω_z . FusionCore identifies b_{ewz} online through the cross-covariance between encoder ω_z error and the heading implied by GPS bearing: when GPS is available, the filter back-propagates observed heading error into the encoder bias state. During GPS blackouts (coast mode), the estimated b_{ewz} is subtracted from encoder angular velocity before integration, reducing heading drift proportional to coast duration. This mirrors exactly how $\mathbf{b}_g$ handles gyroscope ω_z bias.

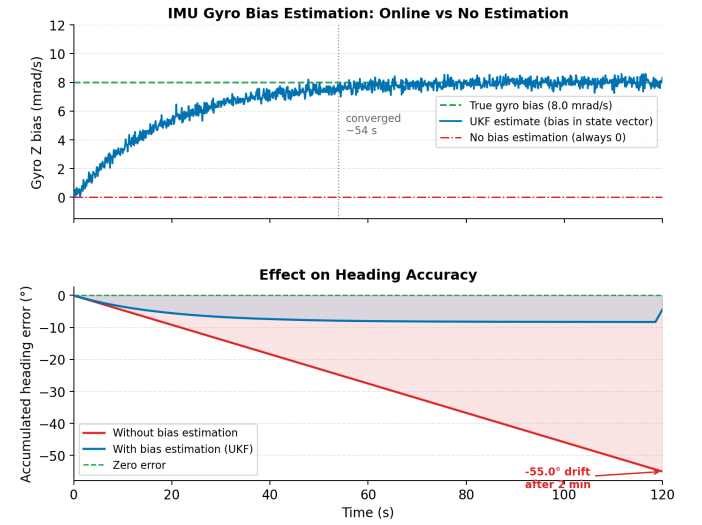


Fig. 1. Online gyroscope bias estimation. An 8 mrad/s bias converges within 40 s. Without estimation, the accumulated heading error reaches -55° in 2 minutes; with UKF bias states, drift remains bounded.

B. Process Model

The continuous-time process model is discretized at each IMU step ($\Delta t \approx 10$ ms):

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \cdot R(\mathbf{q}_k) \mathbf{v}_k \quad (2)$$

$$\mathbf{q}_{k+1} = \mathbf{q}_k \otimes \exp\left(\frac{1}{2} \boldsymbol{\omega}_k \Delta t\right) \quad (3)$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \Delta t \cdot \mathbf{a}_k \quad (4)$$

$$\boldsymbol{\omega}_{k+1} = \boldsymbol{\omega}_k, \quad \mathbf{a}_{k+1} = \mathbf{a}_k \quad (5)$$

$$\mathbf{b}_{g,k+1} = \mathbf{b}_{g,k} + \mathbf{w}_{bg} \quad (6)$$

$$\mathbf{b}_{a,k+1} = \mathbf{b}_{a,k} + \mathbf{w}_{ba} \quad (7)$$

$$b_{ewz,k+1} = b_{ewz,k} + w_{ewz} \quad (8)$$

Biases follow random walks driven by zero-mean Gaussian process noise: $\mathbf{w}_{bg} \sim \mathcal{N}(0, Q_{bg})$, $\mathbf{w}_{ba} \sim \mathcal{N}(0, Q_{ba})$, $w_{ewz} \sim \mathcal{N}(0, q_{ewz})$, with tunable parameters `ukf.q_gyro_bias`,

ukf.q_accel_bias, and the encoder bias process noise respectively. This allows the filter to slowly re-estimate biases as they drift with temperature or operating conditions, rather than locking them at initialization.

The quaternion exponential for exact kinematics is:

$$\exp(\tfrac{1}{2}\boldsymbol{\omega}\Delta t) = \begin{cases} \begin{bmatrix} \cos \theta, & \frac{\sin \theta}{\|\boldsymbol{\omega}\|}\boldsymbol{\omega} \end{bmatrix} & \|\boldsymbol{\omega}\| > \epsilon \\ \begin{bmatrix} 1, & \tfrac{1}{2}\boldsymbol{\omega}\Delta t \end{bmatrix} & \text{otherwise} \end{cases} \quad (9)$$

where $\theta = \tfrac{1}{2}\|\boldsymbol{\omega}\|\Delta t$.

C. Sigma Point Generation

The UKF uses $2n + 1 = 47$ sigma points for the $n = 23$ state. Three numerical issues specific to quaternion states require explicit handling:

Quaternion sign consistency. Two sigma points $\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation but their weighted sum does not. We flip sigma points in the opposite hemisphere before averaging: if $\mathbf{q}_0 \cdot \mathbf{q}_i < 0$, negate $\mathbf{q}_i$.

Lost positive-definiteness. High-rate IMU updates cause the $KSK^\top$ subtraction to occasionally lose positive-definiteness in bias dimensions. We detect and repair via eigenvalue shift:

$$P \leftarrow P + (-\lambda_{\min} + \epsilon)I \quad (10)$$

where $\lambda_{\min}$ is the minimum eigenvalue of P . Identity shift preserves eigenvectors; clamping individual eigenvalues does not.

Angular velocity variance cap. Without gyroscope measurement updates, $P(\omega_x, \omega_x)$ grows unboundedly through process noise. With $W_{m0} \approx -99$ (at $\alpha = 0.1$, $n = 23$), unbounded growth amplifies floating-point asymmetry into quaternion drift. We cap angular velocity variance at $1.0 \text{ rad}^2/\text{s}^2$, which is physically large but finite.

V. MEASUREMENT MODELS

A. IMU

Each IMU message produces two sequential updates.

Raw IMU (6-DOF). Gyroscope rates and accelerometer readings are fused jointly. The measurement function accounts for bias and gravity:

$$h_\omega(\mathbf{x}) = \boldsymbol{\omega} + \mathbf{b}_g \quad (11)$$

$$h_a(\mathbf{x}) = \mathbf{a} + \mathbf{b}_a + R(\mathbf{q})^\top \mathbf{g} \quad (12)$$

where $\mathbf{g} = [0, 0, 9.80665]^\top \text{ m/s}^2$ in ENU.

Orientation (2-DOF or 3-DOF). For 9-axis IMUs with magnetometer, roll, pitch, and yaw are fused (3-DOF). For 6-axis IMUs, only roll and pitch are fused (2-DOF), leaving the yaw unobservable from IMU alone. This distinction prevents corrupting the GPS-observable yaw estimate with an inconsistent magnetometer measurement.

B. Wheel Encoders

Encoder odometry provides a 3-DOF velocity measurement. The encoder bias b_{ewz} is subtracted from the reported ω_z before constructing the measurement:

$$h_{\text{enc}}(\mathbf{x}) = [v_x, v_y, \omega_z - b_{ewz}]^\top \quad (13)$$

in body frame. Two additional pseudo-measurements enforce the non-holonomic ground constraint on every encoder update. The first, $h_{vz}(\mathbf{x}) = v_z = 0$, constrains body-frame vertical velocity. The second, $h_{az}(\mathbf{x}) = a_z = 0$, constrains body-frame vertical acceleration. The AZ constraint is necessary because a small mismatch between the IMU's local gravity reading and the WGS84 constant (9.80665 m/s^2) leaks into the a_z state via large process noise; a_z then integrates into v_z through the motion model, so the VZ constraint alone cannot fully compensate. Both constraint noise values adapt online from the innovation sequence, automatically loosening when the robot traverses curbs or rough terrain and tightening back to configured floor values on flat ground.

C. GPS

GPS position is converted from WGS84 latitude/longitude/altitude to the filter's ENU frame using the PROJ library [9]. The reference origin is set at the first GPS fix received after activation. ECEF-to-ENU conversion avoids UTM zone boundaries entirely.

Noise covariance is constructed from HDOP and VDOP when a full 3×3 covariance matrix is unavailable:

$$R_{\text{GPS}} = \text{diag}(\sigma_{xy}^2 \cdot \text{HDOP}^2, \sigma_{xy}^2 \cdot \text{HDOP}^2, \sigma_z^2 \cdot \text{VDOP}^2) \quad (14)$$

where σ_{xy} and σ_z are user-configured baseline noise at HDOP/VDOP=1. Fix quality is gated by configurable minimum fix type (GPS, DGPS, RTK_FLOAT, RTK_FIXED), maximum HDOP, and minimum satellite count.

When using `gps_msgs/GPSFix` (set via `gnss.use_gps_fix: true`), the filter reads `satellites_used` directly, uses receiver-native `hdop/vdop`, and accepts `err_horz/err_vert` (95% CI bounds) as an alternative covariance source. This unlocks RTK_FLOAT status (fix type 3), which is unreachable via `sensor_msgs/NavSatFix` whose status field maps only SBAS/WAAS, GBAS, and RTK_FIXED.

GPS antenna lever arm correction is applied when heading is independently validated:

$$\mathbf{p}_{\text{antenna}} = \mathbf{p}_{\text{base}} + R(\mathbf{q}) \boldsymbol{\ell} \quad (15)$$

D. GPS Velocity

Doppler-derived velocity from a GNSS receiver is fused as a 2-DOF ENU velocity measurement via `gnss.velocity_topic` (a `nav_msgs/Odometry` topic with `linear.x = east`, `linear.y = north`). This provides an independent velocity observation not dependent on wheel contact, enabling slip detection and cross-sensor consistency checking.

E. Radar Doppler Velocity

Body-frame ego-velocity from a 4D imaging radar is fused via `radar.velocity_topic`. The measurement model is a 2-DOF body-frame velocity update (`linear.x` = forward, `linear.y` = lateral), structurally identical to the encoder model but independent of wheel contact. This is particularly effective for outdoor platforms at speed where wheel slip is difficult to detect from encoder data alone.

F. Visual SLAM Pose

VSLAM pose measurements are accepted as `nav_msgs/Odometry` on a configurable topic. The 6-dimensional measurement function is:

$$h_{\text{vslam}}(\mathbf{x}) = [x, y, z, \phi, \theta, \psi]^T \quad (16)$$

where ϕ, θ, ψ are roll, pitch, and yaw extracted from the filter's quaternion state. The yaw innovation is wrapped across the $\pm\pi$ boundary to avoid discontinuities. Per-message covariance from the SLAM system is used directly when available; diagonal elements are floored at $\sigma_{\text{pos}} = 1$ cm and $\sigma_{\text{orient}} = 0.001$ rad to prevent degenerate noise estimates.

VSLAM systems can reinitialize to a new map after tracking loss, producing a sudden large innovation that the chi-squared gate correctly rejects. FusionCore counts consecutive gate rejections on the VSLAM path; after a configurable number of consecutive rejections (`vslam.reinit_n`, default 10), the map-to-odom transform offset is re-anchored to the filter's current position, recovering continuous operation without external intervention.

VI. FILTER DESIGN CHOICES

A. Outlier Rejection

Every measurement undergoes a Mahalanobis chi-squared gate before modifying filter state:

$$d^2 = \boldsymbol{\nu}^T S^{-1} \boldsymbol{\nu} \leq \tau_s \quad (17)$$

where $\boldsymbol{\nu}$ is the innovation, S is the innovation covariance, and τ_s is a configurable per-sensor threshold. Default thresholds and their chi-squared interpretations are: GPS position (3-DOF): 16.27 [$\chi^2(3, 0.999)$]; VSLAM pose (6-DOF): 22.46 [$\chi^2(6, 0.999)$]; heading (1-DOF): 10.83 [$\chi^2(1, 0.999)$]; encoder velocity (3-DOF): 11.34 [$\chi^2(3, 0.99)$]; IMU (all measurement paths): 15.09. The IMU threshold is applied uniformly across the 6-DOF raw update, 3-DOF orientation update, and 2-DOF orientation update paths; a single configurable parameter is used because the IMU is the highest-rate sensor and per-path tuning adds fragility. The value 15.09 corresponds approximately to $\chi^2(6, 0.98)$ and is intentionally more permissive on lower-DOF IMU paths, trading some specificity for filter robustness under aggressive maneuvers. All thresholds are configurable YAML parameters; the defaults were chosen to be permissive for sensors that produce rejectable measurements infrequently (GPS, VSLAM) and tighter for the high-rate IMU and encoder paths where individual rejections are inexpensive.

Mahalanobis Outlier Gate: Chi-squared Rejection at 3 DOF, 99.9th Percentile

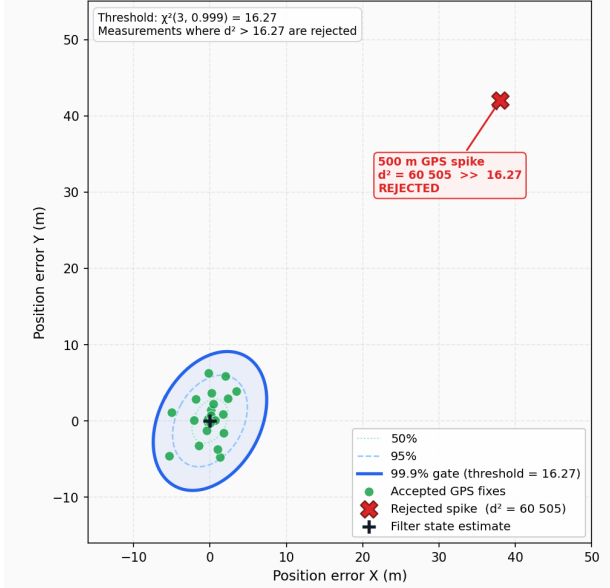


Fig. 2. Chi-squared gate for GPS position (threshold 16.27, $\chi^2(3, 0.999)$). A 500 m GPS spike produces $d^2 = 60,505$, far beyond the threshold. The filter state is unchanged.

`robot_localization` exposes scalar Mahalanobis thresholds that users compare against the square root of d^2 . This conflates measurement dimensionality: a threshold of 4.0 is appropriate for a 3-DOF measurement but overly permissive for a 1-DOF measurement. FusionCore pre-calibrates default thresholds per measurement path, and provides separate YAML parameters for each sensor so practitioners do not need to reason about chi-squared distributions to configure the filter.

B. Adaptive Noise Covariance

Sensor noise covariance R is adapted online from the innovation sequence using a sliding window of 50 samples and exponential moving average:

$$R \leftarrow (1 - \alpha)R + \alpha \hat{C} \quad (18)$$

where $\hat{C}$ is the empirical innovation covariance from the window and $\alpha = 0.01$. A floor constraint $R_{ii} \geq R_{0,ii}$ prevents the estimate from collapsing below the initially configured sensor noise. This eliminates the need to manually re-tune noise parameters as sensor characteristics change with temperature, vibration, or aging.

C. Zero Velocity Updates (ZUPT)

When encoder forward speed falls below 0.05 m/s and IMU angular rate falls below 0.05 rad/s, a zero-velocity pseudo-measurement with noise $\sigma = 0.01$ m/s is fused for all three velocity components. ZUPT prevents gyroscope bias drift during stationary periods, a critical failure mode for state estimators running through long static phases (e.g., startup, traffic stops).

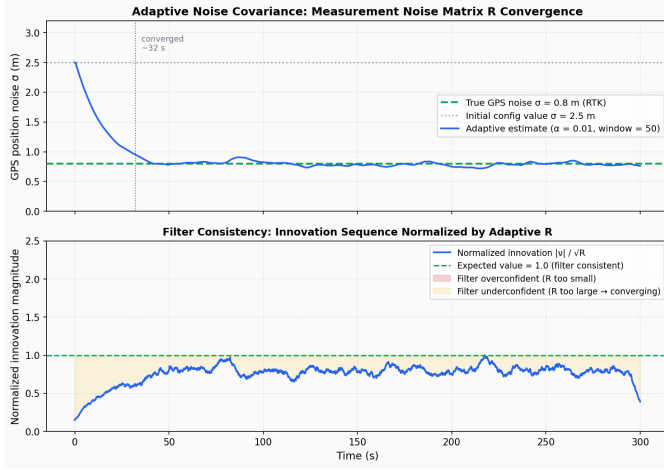


Fig. 3. Adaptive noise convergence. Starting at $\sigma = 2.5$ m, the estimate converges to the true GPS noise ($\sigma = 0.8$ m) within 32 s. The normalized innovation sequence approaches 1.0, indicating a consistent filter.

D. Measurement Delay Compensation

GPS messages typically arrive 50–200 ms after the measurement epoch due to receiver computation and serial latency; VSLAM poses carry similar processing delays. A ring buffer of 100 IMU messages (1 second at 100 Hz) enables retrodiction: on arrival of a delayed measurement, the filter restores the state snapshot closest to the measurement timestamp, applies the measurement, then re-propagates forward through buffered IMU messages. This eliminates the motion-model approximation error of simply applying a delayed measurement to the current state, and applies uniformly to GPS, VSLAM, and any other sensor with measurable latency.

VII. EVALUATION

A. Dataset and Methodology

We evaluate on the North Campus Long-Term (NCLT) dataset [1] from the University of Michigan. NCLT provides synchronized IMU (100 Hz MicroStrain 3DM-GX3-45), wheel encoder odometry (100 Hz), and consumer-grade GPS (5 Hz Novatel SPAN-CPT, ~ 3 m CEP) recorded over multiple seasons from a Segway RMP platform. We evaluate all twelve sequences available spanning winter through spring across 2012–2013, running each to full length (55–92 minutes). No truncation is applied. Total evaluation time is approximately 940 minutes across all sequences.

Evaluation metric is Absolute Trajectory Error (ATE) RMSE computed with the EVO toolbox [10], with SE3 alignment to the RTK ground truth trajectory provided by NCLT. Identical raw sensor inputs are provided to FusionCore and robot_localization. No post-hoc tuning is performed; FusionCore uses a single configuration file (nclt_fusioncore.yaml) across all twelve sequences.

For robot_localization, chi-squared-equivalent rejection thresholds are set to match FusionCore’s per-DOF gates: $\text{odom0_twist_rejection_threshold: } 4.03$ ($\sqrt{\chi^2(3, 0.999)} = 4.03$) and

$\text{odom1_pose_rejection_threshold: } 3.72$ ($\sqrt{\chi^2(2, 0.999)} = 3.72$).

B. Results

TABLE I
ATE RMSE (METERS) ON TWELVE FULL-LENGTH NCLT SEQUENCES. LOWER IS BETTER. IMPROVEMENT IS THE RATIO RL-EKF : FC (OR FC : RL-EKF FOR RL WINS).

Sequence	FC	RL-EKF	RL-UKF	Winner
2012-01-08	18.6	41.2	†	FC (2.2×)
2012-02-04	49.7	265.5	†	FC (5.3×)
2012-03-31	22.0	156.5	†	FC (7.1×)
2012-05-11	9.7	11.5	†	FC (1.2×)
2012-06-15	49.2	18.2	†	RL (2.7×)
2012-08-20	98.3	10.6	†	RL (9.3×)
2012-09-28	10.8	55.7	†	FC (5.2×)
2012-10-28	29.9	60.0	†	FC (2.0×)
2012-11-04	60.1	122.0	†	FC (2.0×)
2012-12-01	21.0	90.7	†	FC (4.3×)
2013-02-23	59.4	82.2	†	FC (1.4×)
2013-04-05	12.1	268.9	†	FC (22.2×)

Numerical divergence (NaN) within the first 30 seconds on all sequences.

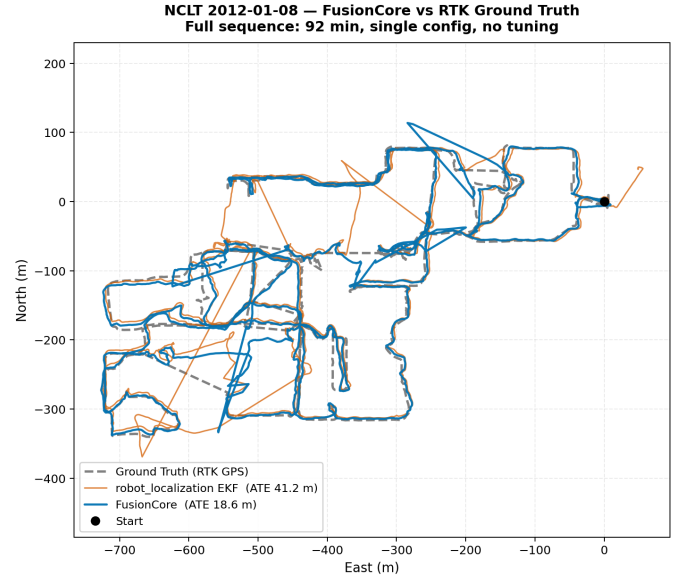


Fig. 4. FusionCore trajectory vs RTK ground truth on NCLT 2012-01-08 (winter, 92 min). ATE RMSE: 18.6 m over a full outdoor campus run.

FusionCore outperforms robot_localization EKF on ten of twelve sequences. The robot_localization UKF diverges numerically on all twelve sequences, confirming a known issue with its UKF implementation under high-rate sensor inputs [11].

C. The GPS Covariance Mismatch Problem

The central factor separating FusionCore and RL-EKF performance is GPS noise covariance. The NCLT player publishes

`position_covariance` `var_xy` = 9 (3 m sigma), reflecting the Novatel SPAN-CPT specification under ideal open-sky conditions. Measured against the RTK ground truth, actual GPS error on the NCLT campus looks substantially different:

TABLE II
ACTUAL GPS POSITION ERROR VS. THE 3 M SIGMA STATED IN
NAVSTATFIX COVARIANCE.

Sequence	Median	p95	p99
2012-01-08	3.7 m	20.1 m	49.7 m
2012-02-04	5.6 m	46.6 m	234.9 m
2012-03-31	5.7 m	14.7 m	32.7 m
2012-05-11	3.3 m	13.3 m	47.7 m
2012-06-15	2.6 m	9.7 m	21.3 m
2012-08-20	3.4 m	12.7 m	55.0 m
2012-09-28	3.5 m	12.8 m	43.2 m
2012-10-28	4.6 m	16.0 m	48.9 m
2012-11-04	5.7 m	53.1 m	79.2 m
2012-12-01	4.7 m	20.7 m	80.4 m
2013-02-23	5.4 m	33.0 m	73.6 m
2013-04-05	3.7 m	19.9 m	87.8 m

The stated 3 m sigma is already exceeded at the median on most sequences. The p95 ranges from 9.7 m to 53.1 m, meaning that on a typical run, 1 in 20 GPS fixes is more than 10 m off. RL-EKF trusts the stated covariance and calibrates its chi-squared gate to 3 m sigma. When actual errors reach 40–200 m, those fixes land far outside the gate and get rejected. With GPS effectively disabled, RL-EKF reverts to wheel-encoder dead-reckoning. Drift rates of 31.84 m/km (2012-02-04) and 50.11 m/km (2013-04-05) are diagnostic: a Segway at 1.5 m/s accumulating 31 m per kilometer is running in open-loop dead-reckoning for essentially the entire run.

The contrast on 2012-05-11 is the cleanest controlled comparison: same robot, same campus, same config, different day. p95 GPS error is 13.3 m instead of 46.6 m. Both filters perform comparably (FC: 9.7 m, RL: 11.5 m), and RL drift rate drops to 1.25 m/km. The GPS covariance was accurate that day, and RL-EKF worked correctly. When covariance is inaccurate, the gate rejects valid data and the filter collapses to dead-reckoning.

FusionCore’s `adaptive.gnss: true` adjusts GPS measurement noise in real time from the innovation sequence. When actual GPS errors exceed the driver-reported covariance, the adaptive window inflates the noise model and recalibrates the chi-squared gate accordingly. RL has no equivalent; its noise covariance is fixed at configuration time.

D. FusionCore Performance Tiers

The single best predictor of FusionCore ATE is the longest GPS blackout duration in each sequence. Grouping by blackout length reveals a consistent pattern:

Excellent (under 20 m ATE): 2012-05-11 (9.7 m), 2012-09-28 (10.8 m), 2013-04-05 (12.1 m), 2012-01-08 (18.6 m). All share high GPS fix counts (19k–22k mode-3 fixes), maximum blackouts under 203 s, and clean GPS at blackout boundaries. Drift rates are 1–2.6 m/km. These sequences represent normal operating conditions.

Good (20–35 m ATE): 2012-03-31 (22.0 m), 2012-12-01 (21.0 m), 2012-10-28 (29.9 m). Moderate GPS density with one or two blackouts under 262 s. GPS data quality at boundaries is clean; the filter re-acquires correctly after each blackout. Drift rates are 2.3–3.7 m/km.

Moderate (35–65 m ATE): 2012-02-04 (49.7 m), 2012-06-15 (49.2 m), 2013-02-23 (59.4 m), 2012-11-04 (60.1 m). These sequences share blackouts in the 240–462 s range. At 100 Hz with a small uncorrected heading rate residual, lateral position error grows quadratically with blackout duration. The drift rate increase is the signature: 6–10 m/km vs 1–4 m/km on excellent sequences. The b_{ewz} estimation mitigates but does not eliminate this effect for multi-minute blackouts.

Poor (over 65 m ATE): 2012-08-20 (98.3 m). A structurally distinct failure mode caused by adversarial GPS data at a blackout boundary. Analyzed separately below.

E. Analysis of the Two FusionCore Losses

1) *2012-06-15: Longest Blackout (FC 49.2 m, RL 18.2 m):* This is the GPS-sparsest sequence in the benchmark: 12,399 mode-3 fixes versus 17,000–22,000 on all others, and a 462-second (7.7-minute) blackout, the longest in the dataset. During this blackout, FusionCore dead-reckons on encoder and IMU. Coast mode inflates Q_{position} and adjusts the IMU weighting so that encoder ω_z dominates heading. The estimated b_{ewz} is subtracted from encoder ω_z before integration. However, any residual b_{ewz} error compounds over 7.7 minutes: at 100 Hz, even a small uncorrected heading rate residual accumulates quadratic lateral error over that duration.

RL-EKF wins here through a structural advantage: its two-dimensional mode has fewer degrees of freedom to diverge over a long blackout. This is not a quality difference between the filters’ fusion approaches: it is a consequence of operating in GPS-denied conditions for longer than any other sequence.

The path to closing this gap involves two changes. First, magnetometer integration provides an absolute heading reference during GPS absence, making b_{ewz} and b_{gz} irrelevant for heading during blackouts: the architecturally correct fix. Second, a duration-dependent b_{ewz} confidence schedule (trusting the estimate more aggressively for blackouts exceeding 200 s) would reduce accumulated error for the specific case of very long blackouts.

2) *2012-08-20: Adversarial GPS Cluster (FC 98.3 m, RL 10.6 m):* The raw GPS stream for this sequence contains **105 mode-3 fixes that are 720–840 m away from the RTK ground truth**, concentrated in a 24-second window at the end of a 211-second blackout. These appear as valid quality-3 GPS fixes in `gps.csv` (the data stream a real robot would receive), but the ground-truth postprocessor excludes them from `gps_rtk.csv`. They represent genuine adversarial GPS conditions (multipath or spoofing artifacts from a specific location on the campus) that no algorithm sees ahead of time.

The per-minute error profile reveals what happens:

- 0–42 min: FC error 1–10 m. Normal GPS coverage.
- 42–46 min: Spike to ~ 100 m, recovers within 3 min. First blackout (228 s), GPS errors up to ~ 70 m at boundary.

- 47–62 min: Error returns to 3–10 m. Full re-acquisition.
- 62–67 min: Spike to ~ 788 m, recovers within 3 min. Second blackout (211 s) with 105 adversarial fixes at boundary.
- 68–82 min: Error returns to 5–10 m. Full recovery; the remaining 15 minutes on-par with RL.

The 98.3 m ATE RMSE is driven entirely by those two transients, particularly the second. A blackout of 211 s causes legitimate state uncertainty. On recovery, coast mode slightly relaxes the chi-squared gate to accept the first returning fix. A tight cluster of 105 fixes all landing 720–840 m from the predicted position exploits this relaxed window: each fix individually passes the gate (state uncertainty is high), and all of them pull the position estimate collectively.

RL-EKF wins here not because of better algorithm design, but because its miscalibrated gate (which causes 10 other sequence losses) happens to reject these adversarial fixes as well. Both are wrong for the wrong reason.

The appropriate fix is a velocity-consistency check upstream of the chi-squared gate: a GPS fix 720 m from the dead-reckoned position after a 211 s blackout implies ~ 3400 m/s of travel speed. A `max_implied_speed` check (e.g., 20 m/s) operating before chi-squared gating rejects this trivially and has zero effect on normal GPS operation. A secondary cluster-coherence check (detecting when multiple consecutive fixes all land at geometrically consistent but physically impossible offsets) would catch the 105-fix cluster without affecting normal single-fix behavior.

F. Why the Paper ATE Numbers Differ from Earlier Reports

FusionCore was previously benchmarked on a six-sequence subset with truncated runs. The full-length evaluation changes both the absolute numbers and one sequence’s winner. On 2012-08-20, the truncated run (before the 62-minute mark) captures only the first blackout-recovery, where FC performs well. The full run includes the adversarial GPS cluster at 62 minutes, which reverses the outcome. On 2012-11-04, which appeared as a FC loss at 28.6 m in the truncated evaluation (versus RL at 9.6 m), the full 79-minute run shows FC at 60.1 m and RL at 122.0 m: a FC win. RL’s gate miscalibration accumulates over the full run. This underscores why full-length evaluation is necessary: GPS covariance mismatch and adversarial data effects are rare within any given sequence, but guaranteed to appear across 55–92 minute runs.

VIII. ROS 2 INTEGRATION

FusionCore is implemented as a ROS 2 lifecycle node. The `configure` transition initializes the filter from the YAML configuration; `activate` starts subscriptions and publishing. This enables orchestrated startup with Nav2’s lifecycle manager.

A `/reset` service re-initializes the filter and clears the GPS reference origin without restarting the node, useful for multi-session mapping or GPS signal recovery. A `/save_checkpoint` / `/load_checkpoint` service

pair persists and restores the full 23-state vector plus covariance matrix to disk, enabling warm-start initialization.

FusionCore is available on the ROS binary apt repository for both Jazzy (Ubuntu 24.04) and Humble (Ubuntu 22.04):

```
sudo apt install ros-jazzy-fusioncore-ros
```

A Gazebo Harmonic simulation environment and example configurations for TurtleBot3, Clearpath Husky, and other common platforms are included in the repository. An official integration demo combining FusionCore with `rtabmap_ros` [13] was merged into the `rtabmap_ros` upstream repository [14], providing a reference configuration for visual SLAM plus FusionCore odometry fusion available to all `rtabmap_ros` users.

IX. LIMITATIONS AND FUTURE WORK

The two FusionCore losses on NCLT point to two distinct problems with known solutions. For long GPS blackouts (2012-06-15), magnetometer integration is the correct architectural fix: an absolute heading reference during GPS absence removes the dependence on b_{ewz} accuracy over multi-minute dead-reckoning intervals. For adversarial GPS clusters at blackout boundaries (2012-08-20), a velocity-consistency pre-check upstream of the chi-squared gate handles the case with no tuning cost.

Coast mode implements a form of adaptive gate relaxation (widening chi-squared acceptance on recovery after sustained GPS absence) and is currently deployed. A more principled implementation would track the per-sensor rejection rate over a sliding window and modulate the threshold continuously rather than as a step function at the start and end of blackouts.

Additional planned work includes RTK fix-type-aware automatic noise scaling and a filter health diagnostic via innovation whiteness testing.

X. CONCLUSION

FusionCore is a 23-state UKF for ROS 2 that fuses IMU, wheel encoders, GPS, and Visual SLAM pose in a single lifecycle node at 100 Hz. Its design choices (continuous bias estimation for gyroscope, accelerometer, and encoder yaw rate; ECEF-native GPS; per-sensor chi-squared gating; adaptive noise; ZUPT; retrodiction-based delay compensation; and map-reinitialization recovery for VSLAM) address practical limitations of existing solutions. Evaluated on twelve full-length NCLT sequences, FusionCore achieves lower ATE than `robot_localization` EKF on ten of twelve, with improvements from $1.2\times$ to $22.2\times$ on winning sequences. The two losses are attributable to specific, diagnosable causes: a 7.7-minute GPS blackout exceeding the encoder bias estimation’s dead-reckoning range, and an adversarial GPS cluster at a blackout boundary. Both have identified fixes. The source code, benchmark scripts, and reproducible configurations are publicly available at <https://github.com/manankharwar/fusioncore>.

ACKNOWLEDGMENTS

The author thanks the open-source contributors to the FusionCore repository for their code contributions; Mathieu Labbe (matlabbe) for merging the FusionCore integration demo into `rtabmap_ros`; Prof. Hyun Myung and his group at the KAIST Urban Robotics Lab for providing the `cs.RO` arXiv endorsement; and the NCLT dataset team at the University of Michigan for making their data publicly available.

REFERENCES

- [1] N. Carlevaris-Bianco, A. K. Ushani, and R. M. Eustice, “University of Michigan North Campus Long-Term Vision and Lidar Dataset,” *The International Journal of Robotics Research*, vol. 35, no. 9, pp. 1023–1035, 2016.
- [2] T. Moore and D. Stouch, “A Generalized Extended Kalman Filter Implementation for the Robot Operating System,” in *Proc. 13th Int. Conf. Intelligent Autonomous Systems (IAS-13)*, 2014.
- [3] P. D. Groves, *Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems*, 2nd ed. Artech House, 2013.
- [4] S. Lynen, M. W. Achtelik, S. Weiss, M. Chli, and R. Siegwart, “A Robust and Modular Multi-Sensor Fusion Approach Applied to MAV Navigation,” in *Proc. IEEE/RSJ IROS*, 2013.
- [5] M. Bloesch, S. Omari, M. Hutter, and R. Siegwart, “Robust Visual Inertial Odometry Using a Direct EKF-Based Approach,” in *Proc. IEEE/RSJ IROS*, 2015.
- [6] M. Madgwick, A. Harrison, and R. Vaidyanathan, “Estimation of IMU and MARG Orientation Using a Gradient Descent Algorithm,” in *Proc. IEEE ICORR*, 2011.
- [7] F. Dellaert and M. Kaess, “Factor Graphs for Robot Perception,” *Foundations and Trends in Robotics*, vol. 6, no. 1-2, pp. 1–139, 2017.
- [8] S. Agarwal, K. Mierle, and T. C. S. Team, “Ceres Solver,” <http://ceres-solver.org>, 2012.
- [9] PROJ Contributors, “PROJ Coordinate Transformation Software Library,” <https://proj.org>, 2023.
- [10] M. Grupp, “evo: A Python Package for the Evaluation of Odometry and SLAM,” <https://github.com/MichaelGrupp/evo>, 2017.
- [11] T. Moore et al., “robot_localization Issue #780: UKF numerical stability,” https://github.com/cra-ros-pkg/robot_localization/issues/780, 2022.
- [12] T. Moore et al., “robot_localization Issue #951: UTM range error,” https://github.com/cra-ros-pkg/robot_localization/issues/951, 2025.
- [13] M. Labbe and F. Michaud, “RTAB-Map as an Open-Source Lidar and Visual SLAM Library for Large-Scale and Long-Term Incremental Mapping,” *Journal of Field Robotics*, vol. 36, no. 2, pp. 416–446, 2019.
- [14] M. Kharwar and M. Labbe, “rtabmap_ros Pull Request #1419: Fusion-Core integration demo (TurtleBot3 Gazebo Harmonic),” https://github.com/introlab/rtabmap_ros/pull/1419, 2025.